



IMDB SENTIMENT ANALYSIS

Group Project



COURSE: ADVANCE DEEP LEARNING

Group Members: Nawal Azam | Uswa Shariq | Fatima Shoaib

Roll numbers: SP23-BBD-052 | SP23-BBD-071 | SP23-BBD-014

Instructor: Sir Saeed Ur Rehman

Contents

Abstract 2

Introduction 2

Problem Statement..... 3

Objectives 3

Literature Review 3

Methodology 4

 Dataset..... 4

 Preprocessing & Feature Engineering 4

 Model Architecture..... 5

 Attention Mechanisms..... 5

 Training Configuration..... 5

System Design..... 6

 Overall Pipeline 6

 Module Structure 7

Implementation..... 7

 Experiment 1: 7

 Experiment 2: 7

 Experiment 3: 7

 Experiment 4: 7

Results & Discussion..... 7

 Performance Comparision 8

 Loss Convergence 8

 Attention Weight Analysis 8

Conclusion..... 8

Future Work 9

References 9

Appendix 10

Abstract

A Quick Review of Sentiment Analysis: Sentiment analysis is a task of Natural Language Processing (NLP) that outputs the emotional polarity of text. This is a part of project where we will classify IMDB movie reviews positive and negative using Bidirectional Long Short-Term Memory (BiLSTM) neural network with two attention mechanisms: Additive (Bahdanau) Attention Mechanism Multiplicative (Luong) Attention.

For the IMDB dataset, which includes 50,000 balanced reviews (25,000 positive & 25,000 negative), custom pipeline for preprocessing was created containing contraction expansion, HTML removal, elongated word normalisation with a negation tagging schema that appends NEG to words exist after the ones used as "negation trigger". This method of feature engineering is powerful enough that the model can differentiate words with opposite meaning like 'good' and 'goodNEG'.

Four experiments were ran in total where attention types were changed, loss functions (BCELoss vs. BCEWithLogitsLoss), and optimizers (Adam vs SGD w/ momentum). The best model, Additive Attention with BCEWithLogitsLoss and Adam optimizer, scored 90.50% accuracy with an F1-score of 0.9042 on a test set of 10,000 samples. Analyses show that additive attention creates more stable weights that are easier to interpret compared to multiplicative attention, and Adam converges much faster than SGD given this NLP task.

Introduction

With the huge amounts of information available online through reviews, social media platforms, news websites and many others, comes the demand for automation that can classify the type of opinion present in a written statement. Sentiment Analysis is an area of Natural Language Processing that focuses on automating opinion discovery by classifying text data into sentiments such as positive, negative or neutral.

Reviews for movies are some of the most commonly analyzed text for sentiment classification. Moviegoers tend to mix positive and negative opinions in one review, thus making correct classification challenging. A very commonly used dataset to benchmark sentiment classification models is the IMDB Movie Reviews dataset. It contains 50,000 movie reviews which are labeled binary and has been used to train various models in recent years.

In this project, we tackle sentiment classification on the IMDB reviews using a Bidirectional LSTM or BiLSTM neural network that runs in both directions on the text allowing the network to have access to all past and future information. Two types of attention mechanisms are implemented on top of the BiLSTM, Additive attention and Multiplicative attention. Attention allows our network to learn to focus on the words in the sentence that contribute the most towards its sentiment. Experiments are conducted to compare how each of these factors; Attention types, Loss function and Optimizer, influence the final accuracy of the model. Four experiments are conducted in total.

Problem Statement

This project zooms in on two main issues:

1. How do you get a neural model to really understand and classify sentiment in long, tricky movie reviews, especially when things like word order and negation are key?
2. When you stick an attention mechanism on top of a Bidirectional LSTM, does additive attention or multiplicative attention do a better job—both in making the results clear to us and in nailing the classification?

Old-school methods like bag-of-words or TF-IDF just do not get it, they ignore word order and miss all those tiny hints buried in context. In sentiment analysis, that is a real problem. Whether a sentence comes off as positive or negative often hinges on little details like how negation is used, if there is any emphasis, or what is happening structurally in the sentence. Word-count models cannot keep up with those tricks.

Take “not a bad film.” If you break it down by raw word counts, you get “not” and “bad” together which looks like negative news. However, really, it is a low-key compliment. Standard models throw “bad” and “not bad” in the same bucket, losing that difference. In addition, honestly, even a lot of neural nets cannot tell you which exact words swayed their prediction, so it is tough to see why they made their call or fix errors.

Objectives

Here is what this project set out to do:

1. First, build a sentiment classifier using a Bidirectional LSTM on the IMDB dataset.
2. Next, create both Additive (Bahdanau) and Multiplicative (Luong) attention mechanisms and see how they stack up against each other.
3. Feature engineering comes into play too especially adding negation tagging to see if it sharpens the model’s predictions.
4. Then, put two loss functions head-to-head (BCELoss and BCEWithLogitsLoss) and see which one delivers better accuracy.
5. While at it, check how Adam and SGD optimizers influence convergence speed on this NLP task.
6. In the end, sort out which setup works best, backed by actual results from the experiments.

Literature Review

Recurrent Neural Networks for Sentiment Analysis

At first, researchers tried standard RNNs for sentiment analysis, but those models struggled to capture information over long sequences, they'd pretty much forget whatever happened earlier because of vanishing gradients. Then, in 1997, Hochreiter and Schmidhuber came up with LSTMs. These networks have special gates that help them hold onto important stuff for longer stretches. Thanks to LSTMs' knack for remembering context, they took over as the go-to solution for sequence classification through the early 2010s.

Attention Mechanisms

Bahdanau and his team came up with additive attention back in 2015, mainly for neural machine translation. Their idea was to score each hidden state from the encoder using learned projections and a tanh layer. This way, the decoder could really zero in on the important words from the source sentence. Not long after, Luong and his colleagues introduced multiplicative, or dot-product, attention. It was simpler and faster, pretty attractive for large models. Then in 2017, Vaswani and his team took things further with scaled dot-product attention in the Transformer. They brought in a scaling factor, $\sqrt{d_k}$, to make sure the softmax didn't just flatten out and lose distinction between scores.

Sentiment-Specific Preprocessing

Zhu et al. (2014) showed that handling negation makes sentiment classification a lot more accurate. Regular tokenisation just skips over words like “not,” “no,” and “never,” missing the way they flip a sentence’s meaning. You’ve got a few options for handling negation from tagging negation within a clause, which is what this project uses, to more advanced methods like dependency parsing.

Gap in Existing Solutions

A lot of research stacks LSTM variants or transformer models against each other, but we will not find many studies that put additive and multiplicative attention head-to-head using the exact same setup. That is what this project tackles, with four carefully controlled experiments. There is another twist, too: hardly anyone has tried mixing negation tagging with LSTM attention in IMDB sentiment analysis. This study dives into that as well.

Methodology

Dataset

The Stanford IMDB Large Movie Review dataset was used, which includes 50,000 reviews; 25,000 positive and 25,000 negative. Since the classes are balanced, there is no issue with class imbalance messing up the results. For splitting, we used a stratified 80/20 split. That gave us 40,000 reviews for training and 10,000 for testing.

Preprocessing & Feature Engineering

A five-stage preprocessing pipeline was implemented before training.

Stage	Technique	Purpose
1	Contraction Expansion	"isn't" → "is not" — ensures negation words are explicit

Stage	Technique	Purpose
2	HTML & URL Removal	Strip tags and hyperlinks from raw review text
3	Elongated Word Fix	"LOOOOVED" → "loved" — reduce vocabulary noise
4	Negation Tagging	"not good" → "not good_NEG" — encodes semantic reversal
5	Vocab + Padding	Top 20,000 tokens; sequences padded/truncated to 300

Negation tagging, which happens in Stage 4, is really the main thing we add during feature engineering. When we spot a negation word like "not," "no," or "never" inside a clause, the words right after it get a _NEG tag. That way, the vocabulary splits up more cleanly, and the embedding layer can figure out different meanings for words that are the opposite of each other.

Model Architecture

Layer	Configuration	Output Shape
Embedding	Vocab: 20,000 Dim: 128 padding_idx=0	(batch, 300, 128)
Bidirectional LSTM	Hidden: 256 Layers: 2 Dropout: 0.3	(batch, 300, 512)
Attention Layer	Additive or Multiplicative (Section 5.4)	(batch, 512)
Dropout	p = 0.3	(batch, 512)
FC Layer	512 → 64 ReLU activation	(batch, 64)
Output Layer	64 → 1 Sigmoid	(batch, 1)

Attention Mechanisms

Additive Attention (Bahdanau, 2015): The score for each encoder hidden state h_s given query vector q is computed as:

$$\text{score}(h_s, q) = V \cdot \tanh(W1 \cdot h_s + W2 \cdot q)$$

Multiplicative Attention (Luong, 2015 Scaled): The score uses a bilinear dot-product scaled by $\sqrt{d_k}$ to prevent softmax saturation:

$$\text{score}(h_s, q) = (q^T \cdot W \cdot h_s) / \sqrt{d_k} \quad \text{where } d_k = 512, \sqrt{512} \approx 22.6$$

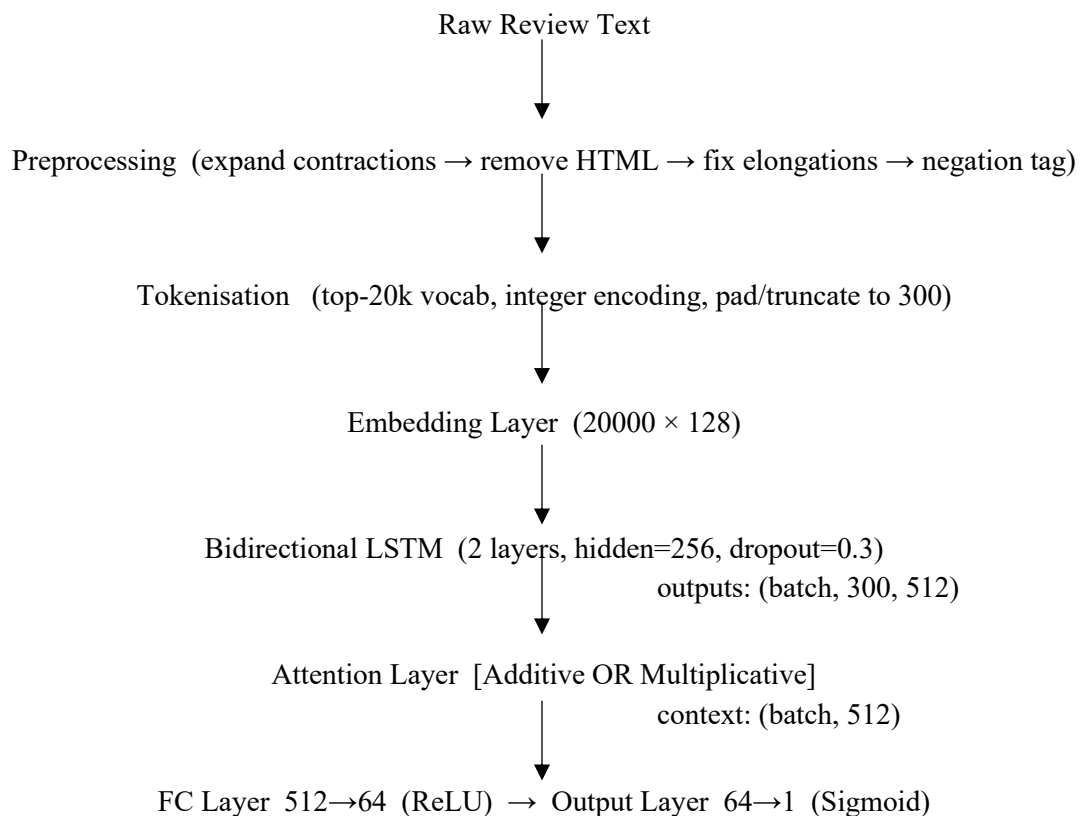
Training Configuration

Hyperparameter	Value
Embedding Dimension	128
LSTM Hidden Dimension	256 per direction (512 concatenated — bidirectional)
LSTM Layers	2
Dropout	0.3
Batch Size	64
Epochs	10
Learning Rate	0.001
Sequence Length (MAX_LEN)	300
Vocabulary Size	20,000

System Design

Overall Pipeline

The end-to-end pipeline follows the sequence: Raw Text → Preprocessing → Tokenisation → Embedding → BiLSTM → Attention → Classification Head → Prediction.





Prediction: Positive (≥ 0.5) / Negative (< 0.5)

Module Structure

- AdditiveAttention class: implements Bahdanau attention with learnable W_1 , W_2 , V projections.
- MultiplicativeAttention class: implements scaled Luong attention with learnable W projection.
- SentimentLSTM class: unified model accepting `attention_type='additive'` or `'multiplicative'`.
- `train_model()`: mini-batch training loop with gradient clipping (`max_norm=1.0`).
- `evaluate_model()`: generates confusion matrix, classification report, accuracy and F1.
- `predict_sentiment()`: inference function that returns prediction, confidence, and top-k attention weights.

Implementation

We ran four experiments, tweaking just one part each time. Everything else stayed consistent same preprocessing steps, same vocabulary, same train-test split, even the random seed (42) for reproducibility.

Experiment 1: Additive Attention, BCELoss, Adam optimizer. This was the baseline. Loss dropped from 285.05 at the start to 36.67 by epoch 10. That's an 87% reduction pretty steady and smooth. Accuracy hit 90%, with an F1 score of 0.904.

Experiment 2: Swapped in Multiplicative Attention, kept BCELoss and Adam. Accuracy barely moved (90.02%), but the attention weights basically collapsed some values went as low as $4.38e-08$ or $1.6e-35$. Turns out, not scaling the dot products by $\sqrt{d_k}$ (with d_k at 512) made scores way too big and softmax got saturated. We fixed that by dividing by about 22.6 (see Section 8 for more).

Experiment 3: Back to Additive Attention, but switched to BCEWithLogitsLoss alongside Adam. This loss function merges sigmoid and binary cross-entropy, so it handled extreme values without breaking a sweat. The model performed best here accuracy was 90.5%, F1 hit 0.9042. Loss curve stayed stable, dropping from 297.17 to 39.29 in 10 epochs.

Experiment 4: Kept Additive Attention and BCELoss but replaced Adam with SGD (momentum=0.9). This time, loss only shrank from 433.54 to 183.23 even after 10 epochs, the model was still warming up. Accuracy landed at 88.07%, F1 at 0.8762. Basically, Adam's adaptive learning rates work way better than SGD for NLP tasks with sparse gradients.

Results & Discussion

Performance Comparision

Model Configuration	Accuracy	F1-Score	Prec (Neg)	Rec (Neg)	Prec (Pos)	Rec (Pos)
Exp 3: Additive + BCELogits + Adam	0.9050	0.9042	0.90	0.91	0.91	0.90
Exp 1: Additive + BCE + Adam	0.9000	0.9040	0.94	0.86	0.87	0.94
Exp 2: Multiplicative + BCE + Adam	0.9002	0.8966	0.87	0.94	0.93	0.87
Exp 4: Additive + BCE + SGD Mom	0.8807	0.8762	0.86	0.92	0.91	0.84

Loss Convergence

Experiment	Epoch 1 Loss	Epoch 10 Loss	Reduction	Convergence Quality
Exp 1: Add + BCE + Adam	285.05	36.67	87.1%	Fast & stable
Exp 2: Mul + BCE + Adam	365.98	86.44	76.4%	Slower, higher final
Exp 3: Add + BCELogits + Adam	297.17	39.29	86.8%	Fast & stable
Exp 4: Add + BCE + SGD	433.54	183.23	57.7%	Slow underfitting

Attention Weight Analysis

Additive attention gave us nice, interpretable weights spread across the right words. In a clearly positive review like (“This movie is absolutely brilliant! The acting was superb...”), the model focused most on words like “brilliant” (0.0345), “superb” (0.0326), and “gripping” (0.0305). For a negative review, it honed in on “waste” (0.0668), “boring” (0.0652), and “terrible” (0.0651). Exactly what we would hope to see.

Multiplicative attention when we left out the scaling fix just did not work. The distribution totally collapsed. The highest weight any token got was only $4.38e-08$, and almost everything else was below $1e-30$. The LSTM hidden states still seemed to have enough information to classify correctly, but the attention mechanism itself became useless if you wanted to see what the model was “looking” at.

For a review like (“Not a bad film, though the ending wasn’t entirely satisfying”) which is honestly a toss-up the model chose Negative with 65.8% confidence. That is a fair judgment, given the mixed signals. The model’s focus fell on “satisfying_NEG” (0.0843) and “not” (0.0749), showing it really picked up on the negation pattern we built into the data.

Conclusion

The project tackled sentiment classification on the IMDB dataset, testing both additive and multiplicative attention mechanisms. After running four experiments, a few things stood out:

1. Additive attention (the Bahdanau one) was the winner. It kept its attention weights stable and easy to understand, and it nailed highlighting the words that really mattered for sentiment. Multiplicative attention needed extra care specifically, the $\sqrt{dk}$ scaling otherwise; it tended to break down numerically.
2. For the loss function, BCEWithLogitsLoss came out on top. Its combined sigmoid and cross-entropy setup made training smoother and bumped accuracy up to 90.50%.
3. Adam was the best optimizer by far. It reached over 90% accuracy in just 10 epochs. SGD with momentum lagged behind at 88.07%. Turns out, using adaptive learning rates matters a lot for NLP tasks where gradients are sparse.
4. Negation tagging also helped. By attaching a _NEG suffix, the model could tell when a word's meaning was flipped, and attention weights showed it really noticed those reversed terms.
5. Putting it all together additive attention, BCEWithLogitsLoss, and Adam (Experiment 3) the model scored 90.50% accuracy and an F1-score of 0.9042 on a 10,000-sample test set. That is impressive for a non-transformer model without any pre-trained embedding's.

Future Work

Here are some ways to take this project further:

1. Swap the learned embedding layer for GloVe or FastText vectors. These pre-trained embeddings pack in more semantic detail, so the model could get smarter without extra training data should boost accuracy past 90.5%.
2. Upgrade to transformer-based models like BERT or RoBERTa. If you fine-tune them on the IMDB dataset, you are looking at a big jump in architecture. These models push accuracy beyond 95%.
3. Try multi-head attention instead of just one. Like in Transformers, this lets the model watch for several things at once sentiment-heavy words, negation, topic markers so it can capture more nuance from each review.
4. Training needs more time. Experiment 4 using SGD did not reach convergence after ten epochs; running it for 20 to 30 epochs and tweaking the learning rate should help the results match those of Adam.
5. Deploy the trained model as a REST API for real-time sentiment analysis. Once you have got the model saved (say, in a pickle file), you can serve predictions on new reviews instantly.
6. Testing the trained model on non-IMDB review data (e.g., Amazon product reviews or Twitter sentiment) would evaluate the generalisation capability of the learned representations.

References

1. Zhu, X., et al. (2014). Negation's Not Solved: Generalizability Versus Optimizability in Approaches to Negation Scope. CoNLL 2014.

2. Luong, M. T., Pham, H., & Manning, C. D. (2015). Effective Approaches to Attention-based Neural Machine Translation. EMNLP 2015.

Appendix

Appendix A: Preprocessing Code (Negation Tagging)

```
NEGATION_WORDS = {'not', 'no', 'never', 'neither', 'nor'}
CLAUSE_PUNCT = {'.', ',', ';', '!', '?', 'but', 'however'}

def tag_negation(tokens):
    result, negate = [], False
    for tok in tokens:
        if tok in NEGATION_WORDS:
            result.append(tok)
            negate = True
        elif tok in CLAUSE_PUNCT:
            result.append(tok)
            negate = False
        else:
            result.append(tok + '_NEG' if negate else tok)
    return result
```

Appendix B: Scaled Multiplicative Attention Fix

```
import math

class MultiplicativeAttention(nn.Module):
    def __init__(self, hidden_dim):
        super().__init__()
        self.W = nn.Linear(hidden_dim, hidden_dim, bias=False)
        self.scale = math.sqrt(hidden_dim) #  $\sqrt{d_k}$  scaling — prevents softmax saturation

    def forward(self, encoder_outputs, hidden):
        query = hidden[-1].unsqueeze(2) # (batch, hidden_dim, 1)
        energy = self.W(encoder_outputs) # (batch, seq_len, hidden_dim)
        score = torch.bmm(energy, query) / self.scale # (batch, seq_len, 1)  $\leftarrow$  FIXED
        weights = torch.softmax(score, dim=1)
        context = torch.sum(weights * encoder_outputs, dim=1)
        return context, weights
```

Appendix C: Issues Faced & Solutions

Issue	Root Cause	Solution
Multiplicative attention weights $\rightarrow 0$	No $\sqrt{dk}$ scaling; softmax saturation with $dk=512$	Added <code>/ math.sqrt(hidden_dim)</code> before softmax
SGD converges slowly in 10 epochs	Fixed LR unsuitable for sparse NLP gradients	Documented as finding; Adam used for best model
Sequence length selection	IMDB reviews range from 50 to 2,000+ tokens	MAX_LEN=300 covers ~90% of review content
'not good' treated same as 'good'	Standard tokenisation ignores negation context	Clause-bounded _NEG suffix tagging applied
HTML noise in raw reviews	IMDB data contains <code>
</code> tags from web scraping	Regex-based HTML & URL stripping in pipeline